

Acodera CRM - Code Quality & Security Analysis

Project: CRM-Acodera (crm-login)

Date: May 18, 2026

Scope: Full-stack review - React frontend, Express API, Supabase Edge Functions, PostgreSQL

Executive Summary

Question Verdict

Is the code neat? ****Partially.**** UI structure is reasonable, but backend/data layers are inconsistent, duplicated, and contain misleading comments and hardcoded secrets.

Is it safe from cyber attacks? ****No - not in current form.**** Critical issues must be fixed before this can be considered productionsafe.

Neatness score: ~5/10 - workable for a small team, not audit-ready.

Security posture: Treat the system as compromised until Supabase keys are rotated and the service-role-in-frontend pattern is removed.

1. Code Neatness

Strengths

- Clear feature-based pages under `src/pages/`
- Shared UI components (Button, Input, dashboard shell)
- Express uses helmet, rate limiting, Zod validation, bcrypt passwords
- Midtrans webhook signature verification
- API keys stored as SHA-256 hashes
- Production Edge API registration requires email verification

Weaknesses

- Two backends (Express + Supabase Edge) that must stay in sync
- Two data paths: REST API vs direct Supabase client in mockData.js (~1,200 lines)
- server/index.js comment claims tenant validation on data routes, but validateTenantAccess is never mounted
- DATABASE_SETUP.md still documents MySQL; actual stack is Supabase Postgres
- Hardcoded Supabase URL/keys in source
- Duplicate AuthProvider in main.jsx and App.jsx
- JavaScript only (no TypeScript) - higher runtime risk

2. Security Assessment

CRITICAL - Fix Immediately

2.1 Service role key exposed in frontend JavaScript

File: src/utils/mockData.js (triggerAutomationEvent, ~lines 1160-1167)

The function sends a Supabase service_role JWT in the Authorization header from the browser when calling trigger-automation.

Impact: Anyone can extract this from the Netlify/JS bundle and gain full database admin access (bypasses all Row Level Security).

Required fix:

- Rotate the Supabase service_role key immediately

- Move automation triggers to server-side only (Edge Function with user JWT or internal secret - never service role in browser)

2.2 Hardcoded Supabase anon key in repository

Files: src/lib/supabase.js, src/utils/templateApi.js

Anon key is embedded as a fallback when env vars are missing.

Impact: Key is public in git and every client build. Combined with weak/misaligned RLS, attack surface expands.

Required fix: Remove hardcoded fallbacks; use env vars only; rotate anon key if repo is public.

HIGH Severity

2.3 OAuth endpoint does not verify OAuth occurred

Files: supabase/functions/api/routes.ts (handleAuthOAuth), server/routes/auth.js

Endpoints accept { email, name } with no Google/Supabase session proof.

Impact: Attacker who knows a victim's email can POST to /api/auth/oauth and receive a valid app JWT (account takeover).

Fix: Verify Supabase Auth ID token server-side before issuing app JWT.

2.4 Client-side tenant isolation only (most CRM data)

File: src/utils/mockData.js (filterBranch)

Filters by branch_id in JavaScript. Supabase client uses anon key and does not attach app JWT to auth.jwt().

RLS policies in supabase-migration-rls-secure.sql use auth.jwt() claims that do not apply to custom JWT in localStorage.

Impact: If RLS is missing, permissive, or includes "OR branch IS NULL", users can query Supabase REST API directly across tenants.

Fix: Route all CRUD through API with server-side tenant checks, OR integrate Supabase-compatible JWT with proper RLS.

2.5 Express contact routes lack branch filtering

File: server/routes/contacts.js

Runs SELECT * FROM contacts with no branch filter despite authenticate middleware.

Impact: IDOR - any authenticated Express user sees all contacts.

Fix: Apply validateTenantAccess or branch filters on every data route.

2.6 External API has no table whitelist

File: supabase/functions/api/routes.ts (handleExternal)

Uses supabase.from(entity) where entity comes from URL path.

Impact: API key holder may access unintended tables (e.g. users).

Fix: Whitelist allowed entities only.

MEDIUM Severity

Issue Location Risk

JWT in localStorage src/utils/auth.jsx XSS leads to session theft

dangerouslySetInnerHTML src/pages/AddInvoice.jsx Stored XSS via invoice HTML templates

Open registration on Express server/routes/auth.js Spam accounts (Edge path has email verify)

CORS origin: true in production server/index.js Reflects any Origin header

AUTOMATION_SECRET optional triggerautomation Edge Function Endpoint callable without auth if unset

Gateway secrets returned to client server/routes/gatewayConfig.js Midtrans server_key exposed to loggedin user /seed admin endpoint supabase/functions/api/index.ts Destructive SQL if SEED_SECRET leaks

3. Security Positives

- Passwords hashed with bcrypt (cost factor 10)
- JWT_SECRET required at Express startup
- Auth rate limiting: 20 attempts per 15 minutes
- Global API rate limiting
- Midtrans webhook signature verification
- API keys hashed with SHA-256
- Netlify Content-Security-Policy headers
- Edge registration requires email verification code
- Audit logging for external API (Edge)

4. Architecture Risk Summary

The application mixes safe patterns (server-side API with service role) with unsafe shortcuts:

- Browser connects to Supabase directly with anon key
- Browser ships service_role JWT for automation triggers
- Attackers follow the shortcuts, not the intended API path

Intended safe path: Browser -> Edge API (service role server-side) -> PostgreSQL

Risky paths: Browser -> Supabase (anon); Browser -> Edge (with leaked service_role)

5. Recommended Remediation Priority

1. Rotate Supabase service_role and anon keys; remove all hardcoded keys from src/
2. Remove service_role from mockData.js; trigger automations server-side only
3. Fix OAuth - verify provider token before issuing JWT
4. Unify data access - browser -> API only, or proper RLS + JWT integration
5. Wire tenant checks on all Express routes; whitelist external API entities
6. Sanitize invoice HTML; avoid dangerouslySetInnerHTML for untrusted content
7. Move JWT to httpOnly cookies (mitigates XSS token theft)
8. Confirm RLS migration is applied in production; test anon access

6. Bottom Line

Neat: Adequate for a prototype; not clean enough for easy security audit.

Safe: No. The exposed service role key in frontend code alone warrants treating the deployment as compromised until keys are rotated and the pattern is removed. Additional risks include OAuth impersonation, client-side-only tenancy, and Express IDOR.

This document is an analysis report only. No application code was modified.